

Helix OTA

Helix OTA — Testing Strategy Document

Document ID: HELOTA-TST-001 **Version:** 1.0.0 **Status:** Active **Last Updated:** 2026-03-05 **Constitution Reference:** HelixConstitution v1 §1, §1.1, §7.1, §11.4.108 **Target Platform:** Android 15 on Orange Pi 5 Max (RK3588)

Table of Contents

1. [Testing Philosophy](#)
 2. [Test Pyramid for Helix OTA](#)
 3. [Server-Side Testing](#)
 4. [Client-Side Testing](#)
 5. [Artifact Validation Testing](#)
 6. [Rollout Testing](#)
 7. [Security Testing](#)
 8. [Hardware-in-the-Loop Testing](#)
 9. [Mutation Testing Strategy](#)
 10. [CI/CD Integration](#)
 11. [Test Data Management](#)
 12. [Acceptance Criteria per HelixConstitution](#)
-

1. Testing Philosophy

1.1 Constitutional Alignment

The Helix OTA testing strategy is governed by the HelixConstitution v1, which mandates four non-negotiable testing principles that form the bedrock of every test we write and every gate we enforce:

1. **§1 — Test Coverage for Every Change:** No code ships without a corresponding test. Coverage is not aspirational; it is a merge gate. A pull request that adds a function without a test is rejected by automation, not by human review.
2. **§1.1 — Mutation Testing:** Line coverage is necessary but not sufficient. A test that executes a line but does not observe its output is a false-positive test. Mutation testing proves that every test is actually testing something by introducing deliberate faults (mutations) into the codebase and verifying that the test suite detects them. Our target: **85% mutation score** across all services.
3. **§7.1 — Anti-Bluff Validation:** Every test must produce evidence that it genuinely exercised the system under test. A test that passes by coincidence — or that would pass regardless of the code under test — is worse than no test at all, because it creates a false sense of security. Anti-bluff validation demands four proofs per test:
 - **Real ACTION:** The test must invoke the actual function, method, or endpoint — not a no-op stub that returns a hardcoded value.
 - **State DELTA:** The test must assert that the system's observable state changed as a result of the action. A test that only asserts "no error occurred" without checking what actually happened fails anti-bluff.
 - **POSITIVE evidence:** The test must produce a concrete artifact (a database row, a log entry, a changed field value) that proves the action succeeded.
 - **Unique evidence TOKEN:** The test must include a unique identifier (e.g., a UUID or timestamp) that ties the evidence to this specific test execution, preventing false passes from stale data or shared state.
4. **§11.4.108 — Four-Layer Fix Verification:** Every bug fix must be verified at four distinct layers before it is considered resolved:

Layer	Name	What It Proves	Example
L1	SOURCE	The code change is correct in isolation	Unit test for the patched function
L2	ARTIFACT	The compiled/built artifact incorporates the fix	Integration test using the built binary
L3	RUNTIME-ON-CLEAN-TARGET	The fix works on a freshly provisioned, clean system	E2E test on a clean Docker environment
L4	USER-VISIBLE	The fix resolves the user-reported symptom	Acceptance test reproducing the original bug report

No bug is closed until all four layers produce a PASS. A fix that works in the source but fails at runtime is not a fix — it is an unverified hypothesis.

1.2 Anti-Bluff Principles in Practice

The anti-bluff mandate (§7.1) requires every test to demonstrate genuine exercise of the system under test. The following table shows how each principle manifests in Helix OTA tests:

Anti-Bluff Pillar	Violation (Bluff)	Compliance (Real)
Real ACTION	Calling a mock that returns a fixed value	Calling the actual <code>UpdateService.CheckForUpdate()</code> with a real database query
State DELTA	Asserting <code>err == nil</code> after a no-op	Asserting that <code>device.Status</code> transitioned from "idle" to "updating" after an update was assigned
POSITIVE evidence	Asserting a function returned <code>true</code>	Asserting a row exists in <code>device_updates</code> with <code>status = "succeeded"</code> and verifying the <code>artifact_id</code> matches
Unique evidence TOKEN	Using a static test device ID	Generating <code>deviceId := fmt.Sprintf("dev_test_%s", uuid.New().String())</code> and

Anti-Bluff Pillar	Violation (Bluff)	Compliance (Real)
	"test-device"	asserting that exact ID appears in the database

Every test in the Helix OTA test suite is audited against these four pillars during code review. A test that fails any pillar is rewritten.

1.3 The Runtime-Signature-as-Definition-of-Done

Per the HelixConstitution, a feature is not “done” when its code compiles and its unit tests pass. A feature is done when its **runtime signature** — the observable, measurable, externally-visible behavior of the running system — matches the specification. The runtime signature is verified at Layer L3 (RUNTIME-ON-CLEAN-TARGET) and Layer L4 (USER-VISIBLE).

For Helix OTA, runtime signatures include:

- A device can check for updates and receive a valid UpdateInfo response with a downloadable artifact URL
- An uploaded artifact passes all four validation stages and its upload_status transitions to "ready" in the database
- A rollout advances from 5% to 10% when the auto-advance health check confirms <5% failure rate
- A device that downloads a tampered artifact rejects it during SHA-256 verification and reports a HASH_MISMATCH telemetry event
- After an A/B update, the device boots into the new slot and reports a commit telemetry event with boot_successful: true

2. Test Pyramid for Helix OTA

The test pyramid defines the distribution of test effort across three layers. The pyramid shape is deliberate: lower-level tests are faster, more deterministic, and cheaper to maintain. Higher-level tests are slower, more complex, and fewer in number.

cycle on real hardware	E2E / System	10% – Full device update
	(Orange Pi 5 Max)	~50 test cases

client-server,	Integration containers (PostgreSQL, etc.)	20% – Database, API, ~200 test cases
	Unit validators, state machines (Pure Go tests)	70% – Business logic, ~700 test cases

2.1 Distribution Targets

Layer	Percentage	Estimated Count	Average Runtime	Total Runtime
Unit	70%	~700	5ms	~3.5s
Integration	20%	~200	500ms	~100s
E2E / System	10%	~50	5min	~250min

2.2 Per-Service Unit Test Count Targets

Service	Target Unit Tests	Rationale
Update Service	100	Cohort assignment, version compatibility, caching logic
Device Service	80	Registration, re-registration, group assignment, credential rotation
Rollout Service	120	Stage progression, health checks, auto-advance, pause/resume/halt
Artifact Service	100	Validation chain, upload pipeline, signing, storage
Telemetry Service	80	Event ingestion,

Service	Target Unit Tests	Rationale
Auth Service	100	aggregation, anomaly detection rules JWT validation, mTLS extraction, RBAC enforcement, TOTP
Notification Service	60	WebSocket hub, event subscription, session management
Client SDK	60	State machine transitions, download manager, verification engine
Total	700	

3. Server-Side Testing

3.1 Unit Tests

Every server service has a corresponding `_test.go` file in its package. Unit tests exercise business logic in isolation by injecting mock implementations of repository interfaces and external dependencies.

Update Service — Cohort Assignment Unit Test:

```
package update
```

```
import (
    "context"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/mock"
    "github.com/stretchr/testify/require"
)
```

```

// MockDeviceRepository is a generated mock for
// DeviceRepository.
type MockDeviceRepository struct{ mock.Mock }

func (m *MockDeviceRepository) GetByID(ctx context.Context,
    id string) (*Device, error) {
    args := m.Called(ctx, id)
    if args.Get(0) == nil {
        return nil, args.Error(1)
    }
    return args.Get(0).(*Device), args.Error(1)
}

func TestCheckForUpdate_CohortAssignment_Deterministic(t
    *testing.T) {
    // ANTI-BLUFF: Unique evidence token per test execution
    deviceID := fmt.Sprintf("dev_test_%s",
        uuid.New().String())

    device := &Device{
        ID:          deviceID,
        Group:        "rk3588_opi5max",
        CurrentVersion: "15.0.0",
    }

    mockDeviceRepo := new(MockDeviceRepository)
    mockDeviceRepo.On("GetByID", mock.Anything,
        deviceID).Return(device, nil)

    // Stub remaining repos – these return "no data" so the
    // test
    // exercises the cohort calculation path.
    mockArtifactRepo := new(MockArtifactRepository)
    mockRolloutRepo := new(MockRolloutRepository)

    svc := &UpdateService{
        deviceRepo:  mockDeviceRepo,
        artifactRepo: mockArtifactRepo,
        rolloutRepo: mockRolloutRepo,
    }

    // ANTI-BLUFF: Real ACTION – invoke the actual method
    result, err := svc.CheckForUpdate(context.Background(),
        deviceID)

    // ANTI-BLUFF: No error expected (no active rollout →
    // nil result)
    require.NoError(t, err)
    assert.Nil(t, result)

    // ANTI-BLUFF: State DELTA – verify the device was
    // actually looked up

```

```

        mockDeviceRepo.AssertCalled(t, "GetByID",
            mock.Anything, deviceID)
    }

    func TestCheckForUpdate_CohortBelowPercentage_ReturnsUpdate(t
        *testing.T) {
        deviceID := fmt.Sprintf("dev_test_%s",
            uuid.New().String())

        // Pick a deviceID whose FNV-32 hash % 100 is < 10
        // We find one by brute force in the test setup
        cohort := fnv32Hash(deviceID) % 100
        if cohort >= 10 {
            t.Skipf("deviceID %s has cohort %d (>= 10),
                skipping", deviceID, cohort)
        }

        device := &Device{
            ID:            deviceID,
            Group:          "rk3588_opi5max",
            CurrentVersion: "15.0.0",
        }
        rollout := &Rollout{
            ID:                "rol_test",
            CurrentPercentage: 10,
            ArtifactID:         "art_test",
        }
        artifact := &Artifact{
            ID:                "art_test",
            TargetVersion:      "15.0.1",
            MinSourceVersion: "15.0.0",
        }

        mockDeviceRepo := new(MockDeviceRepository)
        mockDeviceRepo.On("GetByID", mock.Anything,
            deviceID).Return(device, nil)
        mockRolloutRepo := new(MockRolloutRepository)
        mockRolloutRepo.On("FindActiveForGroup", mock.Anything,
            "rk3588_opi5max").Return(rollout, nil)
        mockArtifactRepo := new(MockArtifactRepository)
        mockArtifactRepo.On("GetByID", mock.Anything,
            "art_test").Return(artifact, nil)

        svc := &UpdateService{
            deviceRepo: mockDeviceRepo,
            artifactRepo: mockArtifactRepo,
            rolloutRepo: mockRolloutRepo,
        }

        // ANTI-BLUFF: Real ACTION
        result, err := svc.CheckForUpdate(context.Background(),
            deviceID)
    }

```



```

require.NoError(t, err)
// ANTI-BLUFF: POSITIVE evidence – result contains
// specific artifact data
require.NotNil(t, result)
assert.Equal(t, "15.0.1", result.Version)
assert.Equal(t, "art_test", result.ArtifactID)
// ANTI-BLUFF: Unique evidence TOKEN – the download URL
// contains the artifact ID
assert.Contains(t, result.DownloadURL, "art_test")
}

```

3.2 Integration Tests with Testcontainers

Integration tests exercise real infrastructure dependencies using testcontainers-go. Each test spawns ephemeral PostgreSQL, MinIO, and Redis containers, ensuring tests run against real database behavior (constraint enforcement, transaction semantics, query performance) rather than in-memory approximations.

```
package integration
```

```

import (
    "context"
    "testing"
    "time"

    "github.com/jackc/pgx/v5/pgxpool"
    "github.com/stretchr/testify/suite"
    "github.com/testcontainers/testcontainers-go"
    tcpostgres "github.com/testcontainers/testcontainers-go/
        modules/postgres"
    "github.com/testcontainers/testcontainers-go/wait"
)

```

```

type DeviceServiceIntegrationSuite struct {
    suite.Suite
    ctx      context.Context
    pgContainer *tcpostgres.PostgresContainer
    pool      *pgxpool.Pool
    deviceSvc *DeviceService
}

```

```

func TestDeviceServiceIntegration(t *testing.T) {
    suite.Run(t, new(DeviceServiceIntegrationSuite))
}

```

```

func (s *DeviceServiceIntegrationSuite) SetupSuite() {
    s.ctx = context.Background()
}

```

```

var err error

s.pgContainer, err = tcpostgres.Run(s.ctx,

```

```

        "postgres:16-alpine",
        tcpostgres.WithDatabase("helix_ota_test"),
        tcpostgres.WithUsername("test"),
        tcpostgres.WithPassword("test"),
        testcontainers.WithWaitStrategy(
            wait.ForLog("database system is ready to accept
connections").
                WithOccurrence(2).
                WithStartupTimeout(30*time.Second)),
    )
    s.Require().NoError(err)

    connStr, err := s.pgContainer.ConnectionString(s.ctx,
        "sslmode=disable")
    s.Require().NoError(err)

    s.pool, err = pgxpool.New(s.ctx, connStr)
    s.Require().NoError(err)

    // Run migrations
    s.runMigrations()

    s.deviceSvc = NewDeviceService(s.pool, nil /* cache */,
        nil /* events */)
}

func (s *DeviceServiceIntegrationSuite) TearDownSuite() {
    s.pool.Close()
    s.pgContainer.Terminate(s.ctx)
}

func (s *DeviceServiceIntegrationSuite)
    TestRegisterDevice_CreatesDatabaseRow() {
    // ANTI-BLUFF: Unique evidence token
    deviceID := fmt.Sprintf("dev_it_%s",
        uuid.New().String())

    req := DeviceRegistrationRequest{
        Serial:          "SN-IT-001",
        Model:           "rk3588_opi5max",
        CurrentVersion:  "15.0.0",
        SlotSuffix:      "_a",
        HardwareFingerprint: "fp_" + uuid.New().String(),
    }

    // ANTI-BLUFF: Real ACTION – hits real PostgreSQL
    device, token, err := s.deviceSvc.RegisterDevice(s.ctx,
        req)

    s.Require().NoError(err)
    s.NotEmpty(token)

```

```

// ANTI-BLUFF: State DELTA – verify the row actually
// exists in the database
var count int
err = s.pool.QueryRow(s.ctx,
    "SELECT COUNT(*) FROM devices WHERE device_id =
    $1", deviceID,
).Scan(&count)
s.Require().NoError(err)
s.Equal(1, count,
    "expected exactly one device row after
    registration")

// ANTI-BLUFF: POSITIVE evidence – verify specific
// fields persisted correctly
var persistedModel string
err = s.pool.QueryRow(s.ctx,
    "SELECT model FROM devices WHERE device_id = $1",
    deviceID,
).Scan(&persistedModel)
s.Require().NoError(err)
s.Equal("rk3588_opi5max", persistedModel)
}

```

3.3 API Contract Tests (Pact-Style)

API contract tests verify that the server and client agree on the shape of API responses. These tests are critical because the Android client and the Go server are developed independently and communicate over a network boundary.

```
package contract
```

```

import (
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestUpdateCheckResponse_Contract(t *testing.T) {
    // Set up a test server that returns a valid UpdateInfo
    // response
    router := setupTestRouter()

    req := httptest.NewRequest(http.MethodGet,
        "/api/v1/devices/dev_contract_test/update-check",
        nil)
    req.Header.Set("Authorization", "Bearer
        "+validTestToken)
}

```

```

w := httptest.NewRecorder()
router.ServeHTTP(w, req)

require.Equal(t, http.StatusOK, w.Code)

// CONTRACT: Verify the response body matches the
// expected schema
var body map[string]interface{}
err := json.Unmarshal(w.Body.Bytes(), &body)
require.NoError(t, err)

// Required fields must be present and non-null
requiredFields := []string{
    "artifact_id", "version", "download_url",
    "sha256", "size_bytes", "signature_url",
}
for _, field := range requiredFields {
    assert.NotNil(t, body[field],
        "contract violation: field '%s' is missing or
        null", field)
}

// Type contracts
assert.IsType(t, "", body["artifact_id"], "artifact_id
    must be a string")
assert.IsType(t, "", body["version"],
    "version must be a string")
assert.IsType(t, "", body["sha256"], "sha256 must be a
    string")
assert.IsType(t, float64(0), body["size_bytes"],
    "size_bytes must be a number")
}

```

3.4 Load/Stress Tests (k6)

Load tests simulate fleet-scale device interactions using [k6](#). The primary scenario simulates 10,000+ devices concurrently checking for updates and downloading artifacts.

```

// k6-load-test.js
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Rate, Trend } from 'k6/metrics';

export const errorRate = new Rate('errors');
export const updateCheckDuration = new
    Trend('update_check_duration', true);
export const downloadDuration = new
    Trend('download_duration', true);

export const options = {
    stages: [

```

```

    { duration: '2m', target:
      1000 }, // Ramp to 1,000 devices
    { duration: '5m', target:
      5000 }, // Ramp to 5,000 devices
    { duration: '5m', target: 10000 }, // Ramp to
      10,000 devices
    { duration: '10m', target: 10000 }, // Sustain
      10,000 devices
    { duration: '3m', target: 0 }, // Ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<500', 'p(99)<2000'], //
      95% < 500ms, 99% < 2s
    errors:
      ['rate<0.01'], // <1%
      error rate
    update_check_duration:
      ['p(95)<200'], // Update checks must be
      fast
  },
};

```

```

const BASE_URL = __ENV.HELOTA_BASE_URL || 'http://
  localhost:8080';

```

```

export default function () {
  const deviceId = `dev_load_${__VU}_${__ITER}`;
  const cert = open(`./testdata/certs/device-${(__VU %
    10)}.pem`);

  // Step 1: Check for updates
  const checkRes = http.get(
    `${BASE_URL}/api/v1/devices/${deviceId}/update-
      check`,
    { headers: { 'Authorization': `Bearer $
      {generateDeviceToken(deviceId)}` } }
  );

  updateCheckDuration.add(checkRes.timings.duration);
  const checkOk = check(checkRes, {
    'update check status is 200 or 204': (r) =>
      r.status === 200 || r.status === 204,
  });
  errorRate.add(!checkOk);

  // Step 2: If update available, download it
  if (checkRes.status === 200) {
    const body = JSON.parse(checkRes.body);
    const dlRes = http.get(`${BASE_URL}$
      {body.download_url}`);
    downloadDuration.add(dlRes.timings.duration);

    const dlOk = check(dlRes, {

```

```

        'download status is 200': (r) => r.status ===
200,
        'download body is not empty': (r) =>
r.body.length > 0,
        'SHA-256 header present': (r) =>
        r.headers['X-Artifact-SHA256'] !==
undefined,
    });
    errorRate.add(!dlOk);
}

sleep(1); // Simulate device processing time
}

```

3.5 Chaos Tests

Chaos tests verify system resilience under failure conditions. These tests inject failures into infrastructure dependencies and assert that the system degrades gracefully rather than crashing or corrupting data.

Chaos Scenario	Injection Method	Expected Behavior	Test Type
Database connection lost	Kill PostgreSQL container mid-request	API returns 503, no data corruption	Integration
Database query timeout	Set <code>statement_timeout = '1ms'</code>	Context cancellation propagates, no goroutine leak	Integration
Object storage unavailable	Block MinIO port via iptables	Upload returns 503, existing artifacts still servable	Integration
Redis cache failure	Flush all keys + set <code>maxmemory 1byte</code>	Cache miss fallback to DB, increased latency but no errors	Integration
Disk full	Fill <code>/tmp</code> partition to 100%	Artifact upload fails with clear error,	Integration

Chaos Scenario	Injection Method	Expected Behavior	Test Type
Network partition	Use tc netem to add 5s latency + 20% packet loss	server remains healthy Client retries with backoff, eventual consistency	E2E
OOM kill	Set container memory limit to 128MB	Server restarts cleanly, in-flight requests fail gracefully	Integration

package chaos

```

func TestDatabaseFailure_UpdateCheckReturns503(t
    *testing.T) {
    ctx := context.Background()

    // Start PostgreSQL container
    pgContainer := startPostgresContainer(t)
    defer pgContainer.Terminate(ctx)

    pool := connectPool(t, pgContainer)
    svc := NewUpdateService(pool, nil, nil, nil, nil)

    // KILL the database mid-operation
    pgContainer.Stop(ctx)

    // ANTI-BLUFF: Real ACTION – attempt to use the service
    // after DB loss
    _, err := svc.CheckForUpdate(ctx, "dev_chaos_test")

    // ANTI-BLUFF: State DELTA – error is returned, not
    // silently swallowed
    require.Error(t, err)
    assert.Contains(t, err.Error(), "device lookup")
}

```

4. Client-Side Testing

4.1 State Machine Unit Tests

The client-side update lifecycle is modeled as a state machine with defined transitions. Every transition must be tested for valid and invalid inputs.

```
package statemachine

import (
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestStateMachine_ValidTransition_IdleToDownloading(t
    *testing.T) {
    // ANTI-BLUFF: Unique evidence token
    token := uuid.New().String()
    sm := NewStateMachine(WithID(token))

    // ANTI-BLUFF: Real ACTION – trigger state transition
    err := sm.Transition(EventUpdateAvailable)

    // ANTI-BLUFF: State DELTA – state changed from Idle to
    // Downloading
    require.NoError(t, err)
    assert.Equal(t, StateDownloading, sm.CurrentState())
}

func TestStateMachine_InvalidTransition_IdleToInstalling(t
    *testing.T) {
    sm := NewStateMachine()

    // ANTI-BLUFF: Real ACTION – attempt invalid transition
    err := sm.Transition(EventInstallComplete)

    // ANTI-BLUFF: POSITIVE evidence – error is returned for
    // invalid transition
    require.Error(t, err)
    assert.Equal(t, StateIdle, sm.CurrentState(),
        "state must not change on invalid transition")
}

// TestStateMachine_AllValidTransitions verifies every legal
// transition.
func TestStateMachine_AllValidTransitions(t *testing.T) {
    tests := []struct {
        name      string
        events    []Event
    }
```



```

        expected State
    }{
        {"idle→downloading", []Event{EventUpdateAvailable},
        StateDownloading},
        {"idle→downloading→verifying",
        []Event{EventUpdateAvailable,
        EventDownloadComplete}, StateVerifying},
        {"idle→downloading→verifying→installing",
        []Event{EventUpdateAvailable,
        EventDownloadComplete, EventVerifyComplete},
        StateInstalling},
        {"idle→downloading→verifying→installing→rebooting",
        []Event{EventUpdateAvailable,
        EventDownloadComplete, EventVerifyComplete,
        EventInstallComplete}, StateRebooting},
        {"full happy path to succeeded",
        []Event{EventUpdateAvailable,
        EventDownloadComplete, EventVerifyComplete,
        EventInstallComplete, EventRebootComplete,
        EventCommitComplete},
        StateSucceeded},
        {"downloading→failed on error",
        []Event{EventUpdateAvailable, EventError},
        StateFailed},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            sm := NewStateMachine()
            for _, event := range tt.events {
                err := sm.Transition(event)
                require.NoError(t, err,
                    "transition on event %v failed", event)
            }
            assert.Equal(t, tt.expected, sm.CurrentState())
        })
    }
}

```

4.2 Verification Engine Unit Tests

The verification engine performs SHA-256 hash comparison and RSA-4096-PSS signature verification. These are security-critical functions that must be tested with both positive and negative cases.

```
package verification
```

```
import (
    "crypto/rand"
    "crypto/rsa"
    "crypto/sha256"
    "os"

```

```

"path/filepath"
"testing"

"github.com/stretchr/testify/assert"
"github.com/stretchr/testify/require"
)

func TestVerifyArtifact_SHA256Match_Succeeds(t *testing.T) {
    // Create a test artifact file with known content
    content := []byte("test artifact payload for SHA-256
        verification")
    artifactPath := filepath.Join(t.TempDir(),
        "test_payload.bin")
    require.NoError(t, os.WriteFile(artifactPath, content,
        0644))

    // Compute expected hash
    hash := sha256.Sum256(content)
    expectedHash := hex.EncodeToString(hash[:])

    // ANTI-BLUFF: Real ACTION – verify the actual file
    err := VerifyArtifactHash(artifactPath, expectedHash)

    // ANTI-BLUFF: State DELTA – no error means hash matched
    require.NoError(t, err)
}

func TestVerifyArtifact_SHA256Mismatch_Fails(t *testing.T) {
    content := []byte("test artifact payload")
    artifactPath := filepath.Join(t.TempDir(),
        "test_payload.bin")
    require.NoError(t, os.WriteFile(artifactPath, content,
        0644))

    wrongHash :=
        "0000000000000000000000000000000000000000000000000000000000000000"

    // ANTI-BLUFF: Real ACTION
    err := VerifyArtifactHash(artifactPath, wrongHash)

    // ANTI-BLUFF: POSITIVE evidence – specific error for
    // hash mismatch
    require.Error(t, err)
    assert.Contains(t, err.Error(), "SHA-256 mismatch")
}

func TestVerifyArtifact_RSASignature_ValidSignature_Succeeds(t
    *testing.T) {
    // Generate RSA key pair for testing
    privateKey, err := rsa.GenerateKey(rand.Reader, 4096)
    require.NoError(t, err)

```

```

content := []byte("test artifact payload for RSA
signature verification")
artifactPath := filepath.Join(t.TempDir(),
    "test_payload.bin")
require.NoError(t, os.WriteFile(artifactPath, content,
    0644))

// Sign the artifact
hash := sha256.Sum256(content)
signature, err := rsa.SignPSS(rand.Reader, privateKey,
    crypto.SHA256, hash[:], &rsa.PSSOptions{
        SaltLength: rsa.PSSSaltLengthEqualsHash,
        Hash:        crypto.SHA256,
    })
require.NoError(t, err)

// ANTI-BLUFF: Real ACTION – verify with the correct
public key
err = VerifyArtifactSignature(artifactPath, signature,
    &privateKey.PublicKey)

require.NoError(t, err)
}

func TestVerifyArtifact_RSARongKey_Fails(t *testing.T) {
    // Sign with one key
    signingKey, _ := rsa.GenerateKey(rand.Reader, 4096)
    // Verify with a different key
    wrongKey, _ := rsa.GenerateKey(rand.Reader, 4096)

    content := []byte("test artifact payload")
    artifactPath := filepath.Join(t.TempDir(),
        "test_payload.bin")
    require.NoError(t, os.WriteFile(artifactPath, content,
        0644))

    hash := sha256.Sum256(content)
    signature, _ := rsa.SignPSS(rand.Reader, signingKey,
        crypto.SHA256, hash[:], nil)

    // ANTI-BLUFF: Real ACTION – verify with wrong public
key
err := VerifyArtifactSignature(artifactPath, signature,
    &wrongKey.PublicKey)

    require.Error(t, err)
    assert.Contains(t, err.Error(), "RSA signature
verification failed")
}

```

4.3 Download Manager Integration Tests

package download

```
import (
    "net/http"
    "net/http/httptest"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestDownloadManager_ResumeFromPartial_Succeeds(t
    *testing.T) {
    // Set up a test server that supports Range requests
    fullPayload := make([]byte, 1024*1024) // 1 MB
    for i := range fullPayload {
        fullPayload[i] = byte(i % 256)
    }

    server := httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
        rangeHeader := r.Header.Get("Range")
        if rangeHeader == "" {
            w.Header().Set("Content-Length",
                fmt.Sprintf("%d", len(fullPayload)))
            w.Header().Set("Accept-Ranges", "bytes")
            w.Write(fullPayload)
            return
        }
        // Parse Range header and respond with 206 Partial
        Content
        // ... (range parsing logic)
        w.Header().Set("Content-Range", fmt.Sprintf("bytes
            %d-%d/%d", start, end, len(fullPayload)))
        w.WriteHeader(http.StatusPartialContent)
        w.Write(fullPayload[start:end+1])
    })))
    defer server.Close()

    dm := &DownloadManager{
        client:    server.Client(),
        storageDir: t.TempDir(),
    }

    // First download: complete
    fullHash := sha256.Sum256(fullPayload)
    expectedSHA256 := hex.EncodeToString(fullHash[:])

    result, err := dm.Download(context.Background(),
        server.URL, expectedSHA256,
        int64(len(fullPayload)), nil)
```

```

    require.NoError(t, err)
    assert.Equal(t, expectedSHA256, result.SHA256)
    assert.Equal(t, int64(len(fullPayload)), result.Size)
}

```

4.4 On-Device Integration Tests (ADB-Connected)

On-device tests run on physical Orange Pi 5 Max hardware connected via ADB. These tests validate the client SDK's interaction with the actual Android update_engine service.

```
// +build ondevice
```

```
package ondevice
```

```
import (
    "testing"
    "time"

    "github.com/stretchr/testify/require"
)
```

```
func TestOnDevice_UpdateCheck_ReachesServer(t *testing.T) {
    checker := NewUpdateChecker(Config{
        ServerURL:    "https://api.staging.helix-ota.io",
        DeviceID:     mustGetDeviceID(t), // Read from
        // system properties
        CheckInterval: 4 * time.Hour,
    })

    // ANTI-BLUFF: Real ACTION – hits the staging server
    // over the network
    info, err := checker.Check(context.Background())

    require.NoError(t, err)
    // Either nil (no update) or valid UpdateInfo – both are
    // acceptable
    if info != nil {
        // ANTI-BLUFF: POSITIVE evidence – verify response
        // fields
        require.NotEmpty(t, info.ArtifactID)
        require.NotEmpty(t, info.DownloadURL)
        require.NotEmpty(t, info.SHA256)
    }
}

```

5. Artifact Validation Testing

5.1 Validation Chain Test Matrix

The artifact validation chain has four stages. Each stage must be tested individually and in combination. The following matrix covers all combinations:

Test Case	Hash	Signature	Structure	Compatibility	Expected Result
V1: All pass	PASS	PASS	PASS	PASS	VALID
V2: Hash fail	FAIL	—	—	—	INVALID (short-circuit at stage 1)
V3: Hash pass, signature fail	PASS	FAIL	—	—	INVALID (short-circuit at stage 2)
V4: Hash+sig pass, structure fail	PASS	PASS	FAIL	—	INVALID (short-circuit at stage 3)
V5: First 3 pass, compat fail	PASS	PASS	PASS	FAIL	INVALID
V6: Empty file	FAIL	—	—	—	INVALID
V7: Non-ZIP file (renamed .zip)	PASS	—	FAIL	—	INVALID
V8: ZIP missing payload.bin	PASS	PASS	FAIL	—	INVALID
V9: ZIP missing care_map.pb	PASS	PASS	FAIL	—	INVALID
V10: Wrong target model	PASS	PASS	PASS	FAIL	INVALID
V11: Downgrade version	PASS	PASS	PASS	FAIL	INVALID

5.2 Negative Tests — Tampered Artifacts

package validation

```

func TestValidationChain_CorruptedPayload_HashFails(t
    *testing.T) {
    // Create a valid artifact, then corrupt one byte
    artifactPath := createTestArtifact(t, validOTAContent)

    // Corrupt byte at offset 1024
    f, err := os.OpenFile(artifactPath, os.O_RDWR, 0644)
    require.NoError(t, err)
    f.WriteAt([]byte{0xFF}, 1024)
    f.Close()

    chain := NewValidationChain(hashChecker, sigChecker,
        structChecker, compatChecker)

    // ANTI-BLUFF: Real ACTION – validate the corrupted file
    result, err := chain.Validate(context.Background(),
        artifactPath, testArtifact)

    require.NoError(t, err)
    // ANTI-BLUFF: State DELTA – validation failed at hash
    // stage
    assert.False(t, result.Valid)
    require.Len(t, result.Stages, 1, "should short-circuit
        after hash failure")
    assert.False(t, result.Stages[0].Passed)
    assert.Equal(t, "hash", result.Stages[0].Name)
}

func TestValidationChain_WrongSigningKey_SignatureFails(t
    *testing.T) {
    // Create artifact signed with key A
    artifactPath := createSignedTestArtifact(t, keyA,
        validOTAContent)

    // Configure validator to verify with key B
    chain := NewValidationChain(
        &HashChecker{},
        &SignatureChecker{PublicKey: keyB.Public()}, //
        // Wrong key!
        &StructureChecker{},
        &CompatibilityChecker{},
    )

    result, err := chain.Validate(context.Background(),
        artifactPath, testArtifact)

    require.NoError(t, err)
    assert.False(t, result.Valid)
    // Stage 1 (hash) should pass, stage 2 (signature)
    // should fail
    require.Len(t, result.Stages, 2)
    assert.True(t, result.Stages[0].Passed, "hash should
        still pass")
}

```

```

    assert.False(t, result.Stages[1].Passed,
        "signature must fail with wrong key")
}

func TestValidationChain_ModifiedPayloadAfterSigning(t
    *testing.T) {
    // Sign the original payload
    artifactPath := createSignedTestArtifact(t, signingKey,
        originalContent)

    // Modify the payload AFTER signing (simulates tampering
        at rest)
    appendToFile(t, artifactPath, []byte("TAMPERED"))

    chain := NewValidationChain(/* ... */)
    result, err := chain.Validate(context.Background(),
        artifactPath, testArtifact)

    require.NoError(t, err)
    assert.False(t, result.Valid, "tampered payload must be
        rejected")
}

```

5.3 Mutation Testing for Validators

Each validator must have mutation pairs. A mutation pair is a test that catches a specific deliberate fault in the validator code.

Validator	Mutation	Catching Test
HashChecker	Change hash.Sum(nil) to hash.Sum([]byte{})	Verify that empty- altered hash produces wrong digest
HashChecker	Remove io.Copy(hash, file)	Hash of empty file should not match artifact hash
SignatureChecker	Change rsa.VerifyPSS to always return nil	Tampered artifact should be rejected
SignatureChecker	Use crypto.SHA1 instead of crypto.SHA256	SHA-1 signatures must be rejected
StructureChecker	Remove "care_map.pb" from required files list	OTA without care_map.pb must fail validation
CompatChecker		

Validator	Mutation	Catching Test
	Change > to >= in version comparison	Downgrade from 15.0.1 to 15.0.1 must be rejected

6. Rollout Testing

6.1 Gradual Rollout Progression Tests

package rollout

```

func TestGradualRollout_StandardProgression_5_10_30_50_100(t
    *testing.T) {
    ctx := context.Background()

    rollout := createTestRollout(t, ctx, "gradual", 5, 5)
    require.Equal(t, 5.0, rollout.CurrentPercentage)

    // Simulate 80% devices reporting SUCCESS at 5%
    simulateDeviceReports(t, ctx, rollout.ID, 80,
        20) // 80% success, 20% in-progress

    // ANTI-BLUFF: Real ACTION – advance the rollout
    advanced, err := rolloutSvc.AdvanceRollout(ctx,
        rollout.ID, 10)
    require.NoError(t, err)
    // ANTI-BLUFF: State DELTA – percentage increased
    assert.Equal(t, 10.0, advanced.CurrentPercentage)

    // Continue progression
    simulateDeviceReports(t, ctx, rollout.ID, 90, 10)
    advanced, err = rolloutSvc.AdvanceRollout(ctx,
        rollout.ID, 30)
    require.NoError(t, err)
    assert.Equal(t, 30.0, advanced.CurrentPercentage)

    simulateDeviceReports(t, ctx, rollout.ID, 95, 5)
    advanced, err = rolloutSvc.AdvanceRollout(ctx,
        rollout.ID, 50)
    require.NoError(t, err)
    assert.Equal(t, 50.0, advanced.CurrentPercentage)

    simulateDeviceReports(t, ctx, rollout.ID, 98, 2)
    advanced, err = rolloutSvc.AdvanceRollout(ctx,
        rollout.ID, 100)
    require.NoError(t, err)
    // ANTI-BLUFF: POSITIVE evidence – rollout completed
    assert.Equal(t, 100.0, advanced.CurrentPercentage)

```

```

    assert.Equal(t, "COMPLETED", advanced.Status)
}

```

6.2 Pause/Resume/Halt Behavior Tests

```

func TestRollout_Pause_StopsAdvance(t *testing.T) {
    ctx := context.Background()
    rollout := createTestRollout(t, ctx, "gradual", 5, 5)

    // ANTI-BLUFF: Real ACTION – pause the rollout
    paused, err := rolloutSvc.PauseRollout(ctx, rollout.ID)
    require.NoError(t, err)
    assert.Equal(t, "PAUSED", paused.Status)

    // Attempt to advance while paused
    _, err = rolloutSvc.AdvanceRollout(ctx, rollout.ID, 10)

    // ANTI-BLUFF: State DELTA – advance is blocked
    require.Error(t, err)
    assert.ErrorIs(t, err, ErrRolloutNotActive)
}

```

```

func TestRollout_Resume_AllowsAdvance(t *testing.T) {
    ctx := context.Background()
    rollout := createActiveThenPausedRollout(t, ctx)

    resumed, err := rolloutSvc.ResumeRollout(ctx,
        rollout.ID)
    require.NoError(t, err)
    assert.Equal(t, "ACTIVE", resumed.Status)

    // ANTI-BLUFF: Real ACTION – advance should now work
    advanced, err := rolloutSvc.AdvanceRollout(ctx,
        rollout.ID, 10)
    require.NoError(t, err)
    assert.Equal(t, 10.0, advanced.CurrentPercentage)
}

```

```

func TestRollout_Halt_CannotBeResumed(t *testing.T) {
    ctx := context.Background()
    rollout := createTestRollout(t, ctx, "gradual", 5, 5)

    halted, err := rolloutSvc.HaltRollout(ctx, rollout.ID,
        "critical failure detected")
    require.NoError(t, err)
    assert.Equal(t, "HALTED", halted.Status)

    // Resume must fail for halted rollouts
    _, err = rolloutSvc.ResumeRollout(ctx, rollout.ID)
    require.Error(t, err)
}

```

```

    assert.ErrorIs(t, err, ErrRolloutHalted)
}

```

6.3 Auto-Rollback Threshold Tests

```

func TestAutoRollback_Exceeds5PercentFailureRate_TriggersRollback(t
    *testing.T) {
    ctx := context.Background()
    rollout := createTestRollout(t, ctx, "gradual", 5, 5)
    rollout.AutoRollbackEnabled = true
    rollout.AutoRollbackThreshold = 0.05

    // Simulate 6% failure rate (above 5% threshold)
    simulateDeviceReports(t, ctx, rollout.ID,
        94, // 94% success
        0,  // 0% in-progress
    )
    // 6% failure rate exceeds threshold

    // ANTI-BLUFF: Real ACTION – attempt advance with high
    // failure rate
    _, err := rolloutSvc.AdvanceRollout(ctx, rollout.ID, 10)

    // ANTI-BLUFF: State DELTA – advance is blocked due to
    // high failure rate
    require.Error(t, err)
    assert.Contains(t, err.Error(), "high failure rate")

    // Verify rollout was automatically rolled back
    updated, _ := rolloutSvc.GetByID(ctx, rollout.ID)
    assert.Equal(t, "ROLLED_BACK", updated.Status)
}

```

6.4 Concurrent Device Update Simulation

```

func TestConcurrentDeviceUpdates_1000Devices(t *testing.T) {
    ctx := context.Background()
    rollout := createTestRollout(t, ctx, "instant", 100, 0)

    // Simulate 1000 devices updating concurrently
    var wg sync.WaitGroup
    errCh := make(chan error, 1000)

    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func(idx int) {
            defer wg.Done()
            deviceID := fmt.Sprintf("dev_concurrent_%d_%s",
                idx, uuid.New().String())
            _, err := deviceSvc.RegisterDevice(ctx,
                DeviceRegistrationRequest{

```

```

        Serial:          fmt.Sprintf("SN-CONC-
%04d", idx),
        Model:           "rk3588_opi5max",
        CurrentVersion:  "15.0.0",
        HardwareFingerprint: fmt.Sprintf("fp_%s",
uuid.New().String()),
    })
    if err != nil {
        errCh <- err
    }
}(i)
}

wg.Wait()
close(errCh)

// ANTI-BLUFF: POSITIVE evidence – no errors during
// concurrent registration
var errors []error
for err := range errCh {
    errors = append(errors, err)
}
assert.Empty(t, errors, "no errors expected during
concurrent device registration")
}

```

7. Security Testing

7.1 Authentication Bypass Tests

```

package security

func TestDeviceEndpoint_NoCertificate_Rejected(t
    *testing.T) {
    router := setupTestRouter()

    req := httptest.NewRequest(http.MethodGet,
        "/api/v1/devices/dev_test/update-check", nil)
    // No client certificate, no Authorization header
    w := httptest.NewRecorder()
    router.ServeHTTP(w, req)

    assert.Equal(t, http.StatusUnauthorized, w.Code)

    var body APIError
    json.Unmarshal(w.Body.Bytes(), &body)
    assert.Equal(t, "MTLS_REQUIRED", body.Code)
}

func TestAdminEndpoint_NoToken_Rejected(t *testing.T) {

```

```

router := setupTestRouter()

req := httptest.NewRequest(http.MethodPost,
    "/api/v1/rollouts", strings.NewReader(`{}`))
req.Header.Set("Content-Type", "application/json")
// No Authorization header

w := httptest.NewRecorder()
router.ServeHTTP(w, req)

assert.Equal(t, http.StatusUnauthorized, w.Code)
}

func TestExpiredJWT_Rejected(t *testing.T) {
    router := setupTestRouter()

    // Generate an already-expired JWT
    expiredToken := generateExpiredJWT(t, "admin")

    req := httptest.NewRequest(http.MethodGet,
        "/api/v1/artifacts", nil)
    req.Header.Set("Authorization", "Bearer "+expiredToken)

    w := httptest.NewRecorder()
    router.ServeHTTP(w, req)

    assert.Equal(t, http.StatusUnauthorized, w.Code)
}

```

7.2 Authorization Boundary Tests

```

func TestViewerCannotCreateRollout(t *testing.T) {
    router := setupTestRouter()
    viewerToken := generateJWT(t, "viewer")

    req := httptest.NewRequest(http.MethodPost,
        "/api/v1/rollouts",
        strings.NewReader(`{"name":"test"}`))
    req.Header.Set("Content-Type", "application/json")
    req.Header.Set("Authorization", "Bearer "+viewerToken)

    w := httptest.NewRecorder()
    router.ServeHTTP(w, req)

    assert.Equal(t, http.StatusForbidden, w.Code)
}

func TestViewerCannotDeleteArtifact(t *testing.T) {
    router := setupTestRouter()
    viewerToken := generateJWT(t, "viewer")

```

```

req := httptest.NewRequest(http.MethodDelete,
    "/api/v1/artifacts/art_test",
    strings.NewReader(`{"reason":"test"}`))
req.Header.Set("Content-Type", "application/json")
req.Header.Set("Authorization", "Bearer "+viewerToken)

w := httptest.NewRecorder()
router.ServeHTTP(w, req)

assert.Equal(t, http.StatusForbidden, w.Code)
}

func TestDeviceCertificateCannotAccessAdminEndpoints(t
    *testing.T) {
    router := setupTestRouter()

    // Create request with device mTLS certificate
    req := httptest.NewRequest(http.MethodGet,
        "/api/v1/users", nil)
    req.TLS = &tls.ConnectionState{
        PeerCertificates:
            []*x509.Certificate{deviceTestCert},
    }

    w := httptest.NewRecorder()
    router.ServeHTTP(w, req)

    assert.Equal(t, http.StatusForbidden, w.Code)
}

```

7.3 Artifact Tampering Detection Tests

```

func TestClientRejectsTamperedArtifact(t *testing.T) {
    // Download a valid artifact
    validArtifact := downloadTestArtifact(t, "art_valid")

    // Tamper with it (flip one byte)
    tamperedPath := tamperWithFile(t, validArtifact, 4096,
        0xFF)

    // ANTI-BLUFF: Real ACTION – verify the tampered
    artifact
    err := VerifyArtifactHash(tamperedPath,
        validArtifactSHA256)

    // ANTI-BLUFF: POSITIVE evidence – hash mismatch
    detected
    require.Error(t, err)
    assert.Contains(t, err.Error(), "SHA-256 mismatch")
}

func TestClientRejectsReplacedSignature(t *testing.T) {

```

```

// Sign with key A, verify with key A – should pass
// Then replace signature with key B's signature –
// should fail

content := []byte("legitimate payload")
sigA := signWithKey(t, keyA, content)
sigB := signWithKey(t, keyB, content)

// Verify with correct signature
err := VerifyArtifactSignature(content, sigA,
    &keyA.PublicKey)
require.NoError(t, err)

// ANTI-BLUFF: Real ACTION – verify with wrong signature
err = VerifyArtifactSignature(content, sigB,
    &keyA.PublicKey)
require.Error(t, err,
    "signature from different key must be rejected")
}

```

7.4 Rate Limiting Effectiveness Tests

```

func TestRateLimiting_DeviceCheckIn_60PerMinute(t
    *testing.T) {
    router := setupTestRouter()
    token := generateDeviceToken(t, "dev_rate_test")

    var rejected int
    for i := 0; i < 70; i++ {
        req := httptest.NewRequest(http.MethodGet,
            "/api/v1/devices/dev_rate_test/update-check",
            nil)
        req.Header.Set("Authorization", "Bearer "+token)

        w := httptest.NewRecorder()
        router.ServeHTTP(w, req)

        if w.Code == http.StatusTooManyRequests {
            rejected++
        }
    }

    // First 60 should succeed, subsequent should be
    // rejected
    assert.GreaterOrEqual(t, rejected, 10,
        "at least 10 requests should be rate-limited after
        60 successful requests")
}

```

8. Hardware-in-the-Loop Testing (Orange Pi 5 Max)

8.1 Test Matrix

The hardware-in-the-loop (HITL) test suite runs on physical Orange Pi 5 Max devices connected via ADB and network. These tests validate the complete update lifecycle on real hardware.

#	Test Case	Update Type	Description	Pass Criteria
H1	A/B update — full OTA	Full A/B	Apply a full OTA package to the inactive slot	Device boots into new slot, slot_suffix changes from _a to _b
H2	Virtual A/B update	Virtual A/B	Apply update using Virtual A/B mechanism	Merge completes, device boots into new version
H3	Recovery update	Recovery	Apply update from recovery partition	Device boots from updated recovery image
H4	Boot verification after update	A/B	Verify dm-verity and AVB pass after update	adb shell dm-verity status returns verified, no boot warnings
H5	Power failure during download	A/B	Cut power at 50% download progress	Device resumes download on reboot, no data corruption
H6	Power failure during install	A/B	Cut power during update_engine apply	Device boots from previous slot

#	Test Case	Update Type	Description	Pass Criteria
				(automatic rollback)
H7	Power failure during reboot	A/B	Cut power during reboot into new slot	Device falls back to old slot on next boot
H8	Rollback verification	A/B	Trigger rollback via dashboard	Device boots from previous slot, version reverts
H9	Download performance	A/B	Measure download speed on Gigabit Ethernet	>50 MB/s sustained throughput
H10	Install time benchmark	A/B	Measure total install time for 1 GB OTA	<5 minutes from download complete to boot into new slot

8.2 Power-Failure Test Automation

```
#!/bin/bash
# hitl_power_failure_test.sh
# Tests device recovery from power loss at various update
# stages

set -euo pipefail

DEVICE_SERIAL=$1
ARTIFACT_VERSION="15.0.1"
POWER_CTRL_IP="192.168.1.100" # IP-controlled power switch

adb_connect() {
    adb connect "$DEVICE_SERIAL"
    adb -s "$DEVICE_SERIAL" wait-for-device
}

power_off() {
    curl -s "http://$POWER_CTRL_IP/off" > /dev/null
}
```

```

        sleep 2
    }

    power_on() {
        curl -s "http://$POWER_CTRL_IP/on" > /dev/null
        sleep 10 # Wait for Android boot
        adb_connect
    }

    echo "=== HITL Test: Power failure during download ==="

    # Start OTA update
    adb -s "$DEVICE_SERIAL" shell am start \
        -n com.helix.ota/.UpdateActivity \
        -a com.helix.ota.START_UPDATE \
        --es artifact_version "$ARTIFACT_VERSION"

    # Wait for download to reach ~50%
    echo "Waiting for download to reach 50%..."
    while true; do
        progress=$(adb -s "$DEVICE_SERIAL" shell \
            "content query --uri content://\
            com.helix.ota.provider/status" | \
            grep -oP 'download_progress=\K[0-9.]+' )
        if ( ( $progress >= 0.50 | bc -l ) ); then
            echo "Download at $progress – cutting power!"
            break
        fi
        sleep 1
    done

    # CUT POWER
    power_off

    # Restore power and verify recovery
    power_on

    # ANTI-BLUFF: State DELTA – device must be in a recoverable state
    status=$(adb -s "$DEVICE_SERIAL" shell \
        "content query --uri content://com.helix.ota.provider/\
        status" | \
        grep -oP 'state=\K[a-z_]+')

    echo "Device state after power recovery: $status"

    if [[ "$status" == "idle" || "$status" == "downloading" ]];
    then
        echo "PASS: Device recovered to '$status' state after power failure"
    else
        echo "FAIL: Device in unexpected state '$status'"
        exit 1
    fi

```

```

fi

# Verify download can resume
adb -s "$DEVICE_SERIAL" shell am start \
    -n com.helix.ota/.UpdateActivity \
    -a com.helix.ota.START_UPDATE \
    --es artifact_version "$ARTIFACT_VERSION"

# Wait for download to complete
echo "Waiting for download to complete..."
timeout=300
while (( timeout > 0 )); do
    state=$(adb -s "$DEVICE_SERIAL" shell \
        "content query --uri content://\
        com.helix.ota.provider/status" | \
        grep -oP 'state=\K[a-z_]+')
    if [[ "$state" == "verifying" || "$state" ==\
        "installing" ]]; then
        echo "PASS: Download resumed and progressed to\
        '$state'"
        break
    fi
    ((timeout--))
    sleep 1
done

```

8.3 Performance Benchmarks

```

// +build hitl

package hitl

import (
    "testing"
    "time"

    "github.com/stretchr/testify/require"
)

func TestDownloadPerformance_GigabitEthernet(t *testing.T) {
    checker := NewUpdateChecker(stagingConfig)
    info, err := checker.Check(context.Background())
    require.NoError(t, err)
    require.NotNil(t, info)

    dm := NewDownloadManager(stagingConfig)

    start := time.Now()
    result, err := dm.Download(context.Background(),
        info.DownloadURL, info.SHA256, info.SizeBytes, nil)
    elapsed := time.Since(start)
}

```

```

require.NoError(t, err)
throughputMbps := float64(info.SizeBytes) /
    elapsed.Seconds() / 1024 / 1024

t.Logf("Downloaded %d bytes in %s (%.1f MB/s)",
    info.SizeBytes, elapsed, throughputMbps)

// ANTI-BLUFF: POSITIVE evidence – throughput exceeds
// minimum
require.Greater(t, throughputMbps, 50.0,
    "download throughput must exceed 50 MB/s on Gigabit
    Ethernet")
}

func TestInstallTime_Benchmark(t *testing.T) {
    // Full install cycle: verify → apply → reboot → commit
    start := time.Now()

    orchestrator := NewInstallOrchestrator(deviceConfig)
    err := orchestrator.ApplyUpdate(context.Background(),
        testArtifactPath)
    require.NoError(t, err)

    err = orchestrator.Reboot(context.Background())
    require.NoError(t, err)

    // Wait for device to come back online
    waitForDeviceOnline(t, 120*time.Second)

    err = orchestrator.Commit(context.Background())
    require.NoError(t, err)

    elapsed := time.Since(start)

    t.Logf("Full install cycle completed in %s", elapsed)
    require.Less(t, elapsed, 5*time.Minute,
        "full install cycle must complete in under 5
        minutes")
}

```

9. Mutation Testing Strategy

9.1 Go-Mutesting Configuration

Mutation testing is performed using [go-mutesting](#), which systematically introduces faults into the codebase and verifies that the test suite detects them.

```

# .mutesting.yaml
target: ./internal/...

```

```

timeout: 30s
workers: 4
verbose: true
min-score: 85

# Exclude generated code and test files
exclude:
  - ".*_mock\\.go"
  - ".*_string\\.go"
  - ".*mock/.*"
  - ".*testdata/.*"

# Mutation operators to use
operators:
  - statement/delete      # Delete individual statements
  - statement/replace     # Replace statement with return
                           values
  - expression/replace    # Replace expressions with
                           constants
  - expression/remove     # Remove expressions (replace
                           with zero values)
  - expression/swap       # Swap binary operands (a+b →
                           b+a)
  - boundary/             # Change boundary conditions (< → <=,
    change                 > → >=)
  - increment/change      # Change increment/decrement (++
    → --)
  - negate/condition      # Negate boolean conditions
  - return/early          # Return early from functions
  - return/value          # Change return values (0 → 1,
                           true → false)

```

9.2 Mutation Operators and Examples

Operator	Mutation	Original Code	Mutated Code	Expected Test Result
statement/ delete	Delete line	cohort := fnv32Hash(deviceID) % 100	(line removed)	Cohort ID is not updated regarding rollout percentage
expression/ replace	Replace with constant	if healthMetrics.FailureRate > 0.05	if false	Auto-rollout test: healthMetrics.FailureRate does not trigger rollback
negate/ condition	Negate boolean	if rollout.Status != "ACTIVE"	if rollout.Status == "ACTIVE"	Pause rollout cannot be triggered

Operator	Mutation	Original Code	Mutated Code	Expected Test Results
return/ value	Change return value	return result.Valid, nil	return true, nil	advanced pause Validation test: i artifacts report valid
boundary/ change	Change boundary	if targetPercentage > 100	if targetPercentage >= 100	Rollout exactl shoul allowe
increment/ change	Swap operands	currentPercentage + incrementStep	incrementStep + currentPercentage	No tes catche (comm — ind test g
return/ early	Return early	func Verify() error { ... }	return nil	Signa test: i signat pass verific

9.3 Per-Service Mutation Score Requirements

Service	Mutation Score Target	Rationale
Update Service	90%	Cohort logic is complex and easy to get wrong
Device Service	85%	Standard CRUD with moderate logic
Rollout Service	90%	Stage progression and health checks are critical
Artifact Service	90%	Validation chain is security-critical
Telemetry Service	80%	Mostly data ingestion with simple aggregation
Auth Service	95%	Security-critical — every auth

Service	Mutation Score Target	Rationale
Notification Service	75%	path must be tested Non-critical path — WebSocket delivery is best-effort
Verification Engine (client)	95%	Cryptographic verification is security-critical
Overall	85%	HelixConstitution §1.1 mandate

9.4 Mutation Testing CI Gate

Mutation testing runs as a nightly CI job. A mutation score below the per-service threshold blocks the merge queue for the next business day.

```
# Run mutation testing with summary report
go-mutesting ./internal/update/... --min-score 90
go-mutesting ./internal/device/... --min-score 85
go-mutesting ./internal/rollout/... --min-score 90
go-mutesting ./internal/artifact/... --min-score 90
go-mutesting ./internal/telemetry/... --min-score 80
go-mutesting ./internal/auth/... --min-score 95
go-mutesting ./internal/notification/... --min-score 75
go-mutesting ./pkg/verification/... --min-score 95
```

10. CI/CD Integration

10.1 GitHub Actions Pipeline

```
# .github/workflows/test.yml
name: Helix OTA Test Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]
  schedule:
    # Nightly: 02:00 UTC
    - cron: '0 2 * * *'
    # Weekly (Sunday): 04:00 UTC – HITL tests
    - cron: '0 4 * * 0'
```

```

env:
  GO_VERSION: '1.22'
  GOLANGCI_LINT_VERSION: 'v1.57'

jobs:
  # -----
  # PRE-MERGE GATES (run on every PR)
  # -----

  unit-tests:
    name: Unit Tests
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: ${ env.GO_VERSION }
      - name: Run unit tests with coverage
        run: |
          go test -v -race -coverprofile=coverage.out
          -covermode=atomic \
            -count=1 -timeout=5m ./internal/... ./pkg/...
      - name: Check minimum coverage (80%)
        run: |
          go tool cover -func=coverage.out | tail -1 | \
            awk '{if ($3+0 < 80) {print "Coverage "$3" < 80%"; exit 1}}'
      - name: Upload coverage to Codecov
        uses: codecov/codecov-action@v4
        with:
          files: coverage.out
          fail_ci_if_error: true

  lint:
    name: Lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: ${ env.GO_VERSION }
      - name: golangci-lint
        uses: golangci/golangci-lint-action@v6
        with:
          version: ${ env.GOLANGCI_LINT_VERSION }
          args: --timeout=5m

  integration-tests:
    name: Integration Tests
    runs-on: ubuntu-latest
    needs: [unit-tests, lint]
    steps:

```


- uses: actions/checkout@v4
- uses: actions/setup-go@v5
- with:
 - go-version: \${{ env.GO_VERSION }}
- name: Run integration tests (testcontainers)
 - run: |
 - go test -v -race -count=1 -timeout=15m
 - tags=integration \
 - ./test/integration/...
- name: Run API contract tests
 - run: |
 - go test -v -race -count=1 -timeout=5m
 - tags=contract \
 - ./test/contract/...

mutation-tests:

```

name: Mutation Tests
runs-on: ubuntu-latest
needs: [unit-tests]
if: github.event_name == 'pull_request'
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-go@v5
    with:
      go-version: ${{ env.GO_VERSION }}
  - name: Install go-mutesting
    run: go install github.com/zimmski/go-mutesting/cmd/go-mutesting@latest
  - name: Run mutation testing (changed packages only)
    run: |
      CHANGED=$(git diff --name-only origin/main...HEAD
      | \
      grep -oP 'internal/\w+' | sort -u | \
      sed 's/^/.\.\/' | sed 's/$/\/...\/' | tr '\n' ' ')
      if [ -n "$CHANGED" ]; then
        go-mutesting $CHANGED --min-score 85
      fi

```

```

# _____
# NIGHTLY JOBS
# _____

```

e2e-tests:

```

name: E2E Tests
runs-on: ubuntu-latest
if: github.event_name == 'schedule'
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-go@v5
    with:
      go-version: ${{ env.GO_VERSION }}
  - name: Run E2E tests against staging

```

```
run: |
  go test -v -count=1 -timeout=60m -tags=e2e \
    ./test/e2e/...
env:
  HELOTA_BASE_URL: https://api.staging.helix-ota.io
```

load-tests:

```
name: Load Tests (k6)
runs-on: ubuntu-latest
if: github.event_name == 'schedule'
steps:
  - uses: actions/checkout@v4
  - name: Install k6
    run: |
      curl https://github.com/grafana/k6/releases/
      download/v0.49.0/k6-v0.49.0-linux-amd64.tar.gz -L |
      \
      tar xvz && sudo mv k6-*/k6 /usr/local/bin/
  - name: Run k6 load test
    run: k6 run --out json=results.json ./test/load/k6-
    load-test.js
  env:
    HELOTA_BASE_URL: https://api.staging.helix-ota.io
  - name: Upload k6 results
    uses: actions/upload-artifact@v4
    with:
      name: k6-results
      path: results.json
```

security-tests:

```
name: Security Tests
runs-on: ubuntu-latest
if: github.event_name == 'schedule'
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-go@v5
    with:
      go-version: ${{ env.GO_VERSION }}
  - name: Run security test suite
    run: |
      go test -v -count=1 -timeout=30m -tags=security \
        ./test/security/...
  - name: Run gosec security scanner
    uses: securego/gosec@master
    with:
      args: '-no-fail -fmt json -out gosec-
      results.json ./...'
  - name: Run nancy for dependency vulnerabilities
    run: |
      go list -json -m all | nancy sleuth
```

full-mutation-testing:

```
name: Full Mutation Testing
runs-on: ubuntu-latest
if: github.event_name == 'schedule'
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-go@v5
    with:
      go-version: ${{ env.GO_VERSION }}
  - name: Install go-mutesting
    run: go install github.com/zimmski/go-mutesting/cmd/
      go-mutesting@latest
  - name: Run full mutation testing
    run: |
      go-mutesting ./internal/... --min-score 85 --html
      mutation-report.html
  - name: Upload mutation report
    uses: actions/upload-artifact@v4
    with:
      name: mutation-report
      path: mutation-report.html
```

```
# _____
# WEEKLY JOBS
# _____
```

```
hitl-tests:
name: Hardware-in-the-Loop Tests
runs-on: [self-hosted, linux, arm64, opi5max]
if: github.event_name == 'schedule' &&
  github.event.schedule == '0 4 * * 0'
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-go@v5
    with:
      go-version: ${{ env.GO_VERSION }}
  - name: Run HITL test suite
    run: |
      go test -v -count=1 -timeout=120m -tags=hitl \
        ./test/hitl/...
  - name: Run power-failure test script
    run: bash ./test/hitl/scripts/power_failure_test.sh
      $DEVICE_SERIAL
  - name: Collect device logs
    if: always()
    run: |
      adb logcat -d > device-logs.txt
  - name: Upload device logs
    if: always()
    uses: actions/upload-artifact@v4
    with:
      name: hitl-device-logs
      path: device-logs.txt
```

```

# -----
# MERGE GATE
# -----

merge-gate:
  name: Merge Gate
  runs-on: ubuntu-latest
  needs: [unit-tests, lint, integration-tests, mutation-
    tests]
  if: always()
  steps:
    - name: Check all pre-merge jobs passed
      run: |
        if [[ "${{ needs.unit-tests.result }}" !=
          "success" || \
            "${{ needs.lint.result }}" != "success" || \
            "${{ needs.integration-tests.result }}" !=
          "success" || \
            "${{ needs.mutation-tests.result }}" !=
          "success" ]]; then
          echo "One or more pre-merge gates failed"
          exit 1
        fi
        echo "All pre-merge gates passed – ready to merge"

```

10.2 Pipeline Summary

Pipeline	Trigger	Jobs	Timeout	Blocking
Pre-merge	Every PR	Unit + Lint + Integration + Mutation	25 min	Yes — blocks merge
Nightly	02:00 UTC daily	E2E + Load + Security + Full Mutation	2 hrs	Yes — creates P0 issue on failure
Weekly	04:00 UTC Sunday	HITL on Orange Pi 5 Max	4 hrs	Yes — creates P1 issue on failure

10.3 Test Result Reporting

All test results are published to:

- **Codecov** — Line coverage trends and per-PR coverage diff

- **GitHub Actions Summary** — Pass/fail with artifact links
 - **Slack #helix-ota-ci** — Real-time notifications for:
 - Pre-merge gate failures (immediate)
 - Nightly test failures (next morning digest)
 - Weekly HITL results (Monday morning report)
 - **Grafana Dashboard** — Historical trends for:
 - Test count over time
 - Coverage percentage trend
 - Mutation score trend
 - Flaky test detection (tests that pass then fail without code changes)
-

11. Test Data Management

11.1 Fixture Generation for OTA Artifacts

Test artifacts are generated programmatically to ensure reproducibility and avoid storing large binary files in the repository.

```
// test/fixtures/artifact.go
package fixtures

import (
    "archive/zip"
    "crypto/rand"
    "crypto/rsa"
    "crypto/sha256"
    "encoding/hex"
    "os"
    "path/filepath"
)

// GenerateOTAArtifact creates a valid OTA ZIP artifact for
// testing.
// The artifact contains:
//   - payload.bin (random data of specified size)
//   - payload_properties.txt (metadata)
//   - care_map.pb (dm-verity care map)
func GenerateOTAArtifact(t testing.TB, sizeBytes int64,
    signingKey *rsa.PrivateKey) (path string,
    sha256Hash string, signature []byte) {
    t.Helper()

    payload := make([]byte, sizeBytes)
    rand.Read(payload)

    tmpDir := t.TempDir()
```

```

artifactPath := filepath.Join(tmpDir, "ota_update.zip")

zipFile, err := os.Create(artifactPath)
require.NoError(t, err)
defer zipFile.Close()

zw := zip.NewWriter(zipFile)

// payload.bin
w, err := zw.Create("payload.bin")
require.NoError(t, err)
_, err = w.Write(payload)
require.NoError(t, err)

// payload_properties.txt
w, err = zw.Create("payload_properties.txt")
require.NoError(t, err)
fmt.Fprintf(w, "FILE_HASH=%s\n", sha256Sum(payload))
fmt.Fprintf(w, "FILE_SIZE=%d\n", sizeBytes)
fmt.Fprintf(w, "METADATA_HASH=%s\n",
    sha256Sum([]byte("metadata")))
fmt.Fprintf(w, "METADATA_SIZE=123\n")

// care_map.pb
w, err = zw.Create("care_map.pb")
require.NoError(t, err)
w.Write([]byte{0x08, 0x01}) // Minimal valid protobuf

require.NoError(t, zw.Close())

// Compute SHA-256
hash := sha256.Sum256(payload)
sha256Hash = hex.EncodeToString(hash[:])

// Sign payload
signature, err = rsa.SignPSS(rand.Reader, signingKey,
    crypto.SHA256, hash[:], nil)
require.NoError(t, err)

return artifactPath, sha256Hash, signature
}

func sha256Sum(data []byte) string {
    h := sha256.Sum256(data)
    return hex.EncodeToString(h[:])
}

```

11.2 Device Simulation Fixtures

```

// test/fixtures/device.go
package fixtures

```

```

type DeviceFixture struct {
    ID            string
    Serial        string
    Model         string
    CurrentVersion string
    SlotSuffix    string
    HardwareFingerprint string
    Group         string
}

func NewDeviceFixture(opts ...DeviceOption) *DeviceFixture {
    id := fmt.Sprintf("dev_fix_%s", uuid.New().String())
    d := &DeviceFixture{
        ID:            id,
        Serial:        "SN-FIX-" + id[4:12],
        Model:         "rk3588_opi5max",
        CurrentVersion: "15.0.0",
        SlotSuffix:    "_a",
        HardwareFingerprint: "fp_" + uuid.New().String(),
        Group:         "rk3588_opi5max",
    }
    for _, opt := range opts {
        opt(d)
    }
    return d
}

type DeviceOption func(*DeviceFixture)

func WithVersion(v string) DeviceOption {
    return func(d *DeviceFixture) { d.CurrentVersion = v }
}

func WithSlotSuffix(s string) DeviceOption {
    return func(d *DeviceFixture) { d.SlotSuffix = s }
}

```

11.3 Database Seeding Scripts

```

// test/seed/seed.go
package seed

// SeedTestDatabase populates the test database with a known
// set of data.
// This function is idempotent – it can be called multiple
// times safely.
func SeedTestDatabase(ctx context.Context, pool
    *pgxpool.Pool) (*SeedData, error) {
    data := &SeedData{}

    // Create device group

```

```

var groupID string
err := pool.QueryRow(ctx,
    `INSERT INTO device_groups (name, description,
        filter_rules)
        VALUES ('rk3588_opi5max', 'Orange Pi 5 Max
        devices', '{"hardware_model":"rk3588_opi5max"}')
        ON CONFLICT DO NOTHING RETURNING id`,
).Scan(&groupID)
if err != nil {
    return nil, fmt.Errorf("seed device group: %w", err)
}
data.DeviceGroupID = groupID

// Create admin user
hashedPassword, _ :=
    bcrypt.GenerateFromPassword([]byte("test-admin-
    password"), 12)
var adminID string
err = pool.QueryRow(ctx,
    `INSERT INTO users (username, email, password_hash,
    role)
    VALUES ('admin@helix.io', 'admin@helix.io', $1,
    'admin')
    ON CONFLICT DO NOTHING RETURNING id`,
    string(hashedPassword),
).Scan(&adminID)
if err != nil {
    return nil, fmt.Errorf("seed admin user: %w", err)
}
data.AdminUserID = adminID

// Create test artifact (metadata only – binary in
// MinIO)
var artifactID string
err = pool.QueryRow(ctx,
    `INSERT INTO artifacts (filename, version, os_type,
    os_version,
        hardware_compatibility, file_size,
        file_hash_sha256, upload_status)
    VALUES ('ota_v15.0.1.zip', '15.0.1', 'android',
    '15',
        '['rk3588_opi5max']', 2147483648,

        'e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855',
        'ready')
    ON CONFLICT DO NOTHING RETURNING id`,
).Scan(&artifactID)
if err != nil {
    return nil, fmt.Errorf("seed artifact: %w", err)
}
data.ArtifactID = artifactID

return data, nil
}

```



```
type SeedData struct {  
    DeviceGroupID string  
    AdminUserID   string  
    ArtifactID    string  
}
```

12. Acceptance Criteria per HelixConstitution

12.1 Four-Layer Test Coverage for Every Feature

Every feature in Helix OTA must have test coverage at all four constitutional layers before it is considered complete:

Layer	Test Type	Example: “Artifact Upload” Feature
L1 — SOURCE	Unit test for ArtifactService.UploadArtifact()	Verify hash computation, validation chain invocation, signature generation
L2 — ARTIFACT	Integration test using the compiled server binary	Start server binary, send multipart upload via HTTP, verify response
L3 — RUNTIME- ON- CLEAN- TARGET	E2E test on fresh Docker environment	Deploy server + PostgreSQL + MinIO from scratch, upload artifact, verify upload_status = 'ready' in database
	Acceptance test matching user story	As an operator, I can upload an

Layer	Test Type	Example: “Artifact Upload” Feature
L4 — USER-VISIBLE		OTA zip file and see it appear as “validated” in the dashboard

A feature that only has L1 and L2 tests is **not done**. The merge gate requires evidence of all four layers.

12.2 Mutation Pair for Every Test

Per HelixConstitution §1.1, every test must have a corresponding mutation pair — a specific mutation that the test catches. This is verified by the mutation testing CI gate. If a test survives all mutations, it is a bluff test and must be rewritten.

Example:

- **Test:**
TestCheckForUpdate_CohortBelowPercentage_ReturnsUpdate
- **Mutation pair:** Delete the line `cohort := fnv32Hash(deviceID) % 100`
- **Expected failure:** Without cohort calculation, all devices (even those with `cohort ≥ rollout percentage`) would receive the update. The test asserts that only devices with `cohort < CurrentPercentage` receive updates, so the mutation is caught.

12.3 Zero Tolerance for False-Positive PASS

A test that passes when it should fail is a **false-positive PASS**. The HelixConstitution mandates zero tolerance for such tests. Prevention mechanisms:

1. **Mutation testing** — Proves that tests can detect real faults
2. **Anti-bluff validation** — Proves that tests observe real state changes
3. **Negative test requirement** — Every feature must have at least one negative test (e.g., “uploading a corrupted artifact returns 422”)

4. **Periodic test suite self-check** — Quarterly, a QA engineer intentionally injects a known bug and verifies the test suite catches it

12.4 Runtime Signature Verification on Clean Target

The ultimate definition of done is the runtime signature verification. For each feature, we define the runtime signature — the observable behavior of the running system — and verify it on a clean target (fresh Docker deployment or factory-reset Orange Pi 5 Max).

Feature	Runtime Signature	Verification Method
Device registration	Device appears in devices table with status = 'online' and correct hardware_model	SQL query on fresh PostgreSQL
Update check	Device receives 200 OK with UpdateInfo JSON containing artifact_id, download_url, sha256	HTTP request to freshly deployed server
Artifact upload	Artifact row transitions to upload_status = 'ready' with 4 artifact_validation_results rows all showing status = 'passed'	Database query after upload to fresh server
Rollout advance	rollouts.current_percentage increases and devices in the new cohort receive updates on next check	End-to-end flow on fresh deployment
A/B update (hardware)	Device boots into new slot with slot_suffix changed and reports commit telemetry event	ADB verification on factory-reset Orange Pi 5 Max

12.5 Compliance Checklist

Before any release is tagged, the following checklist is verified by both the CI system and a human reviewer:

☐

Line coverage $\geq$ 80% across all packages

☐

- ☐ Mutation score $\geq$ 85% across all services ($\geq$ per-service thresholds)
 - ☐ Every feature has L1-L4 test coverage
 - ☐ Every test passes anti-bluff validation (ACTION, DELTA, POSITIVE, TOKEN)
 - ☐ Zero skipped tests without documented justification
 - ☐ Zero flaky tests (3 consecutive nightly runs with no flakes)
 - ☐ All security tests pass (auth bypass, RBAC, artifact tampering, rate limiting)
 - ☐ HITL tests pass on Orange Pi 5 Max (most recent weekly run)
 - ☐ Load test: 10,000 concurrent devices with p99 latency $<$ 2s
 - ☐ No open P0/P1 bugs in the milestone
-

Appendix A: Test Command Reference

Run all unit tests with coverage

```
go test -v -race -coverprofile=coverage.out -count=1  
-timeout=5m ./internal/... ./pkg/...
```

Run integration tests only

```
go test -v -race -count=1 -timeout=15m -tags=integration ./test/integration/...
```

Run security tests

```
go test -v -count=1 -timeout=30m -tags=security ./test/security/...
```

Run E2E tests against staging

```
HELOTA_BASE_URL=https://api.staging.helix-ota.io \  
go test -v -count=1 -timeout=60m -tags=e2e ./test/e2e/...
```

Run HITL tests on connected Orange Pi 5 Max

```
DEVICE_SERIAL=192.168.1.50:5555 \  
go test -v -count=1 -timeout=120m -tags=hitl ./test/hitl/...
```

```

# Run mutation testing for a specific package
go-mutesting ./internal/auth/... --min-score 95

# Run k6 load test
k6 run --out json=results.json ./test/load/k6-load-test.js

# Check coverage threshold
go tool cover -func=coverage.out | tail -1 | awk '{if ($3+0
    < 80) exit 1}'

# Generate test fixtures
go run ./test/fixtures/cmd/generate/main.go --output ./test/
    testdata/

# Seed test database
go run ./test/seed/cmd/seed/main.go --db "postgres://
    test:test@localhost:5432/helix_ota_test"

```

Appendix B: Test Tag Conventions

Go build tags are used to separate test suites by runtime requirements:

Tag	Purpose	Infrastructure Required	Runtime
(none)	Unit tests	None (pure Go)	5s
integration	Integration tests	Testcontainers (Docker)	2min
contract	API contract tests	Running server	30s
security	Security-focused tests	Running server + test certs	5min
e2e	End-to-end tests	Staging environment	30min
hitl	Hardware-in-the-loop	Orange Pi 5 Max + ADB	2hr
load	Load/stress tests	Staging environment + k6	30min

Appendix C: Flaky Test Policy

Flaky tests — tests that pass and fail non-deterministically without code changes — are treated as P1 bugs. The policy is:

1. **Detection:** A test is flagged as flaky if it fails in ≥ 2 out of 10 consecutive nightly runs without a corresponding code change.
2. **Quarantine:** Flaky tests are moved to a `// +build flaky` tag and excluded from the merge gate.
3. **Fix deadline:** Flaky tests must be fixed within 5 business days or they are deleted.
4. **Root cause categories:**
 - Race conditions (missing mutex, goroutine leak)
 - Time-dependent assertions (use `Eventually()` patterns instead of fixed sleeps)
 - Shared state (tests mutating global variables)
 - External service flakiness (use deterministic mocks for unit tests, testcontainers for integration)
 - Resource exhaustion (file descriptor limits, port conflicts)

```
// Example: Replace fixed sleep with Eventually pattern
func TestDeviceUpdatesAfterRegistration(t *testing.T) {
    registerDevice(t, deviceID)

    // BAD: Fixed sleep
    // time.Sleep(2 * time.Second)

    // GOOD: Poll until condition is met
    require.Eventually(t, func() bool {
        device, _ :=
            deviceSvc.GetByID(context.Background(), deviceID)
        return device.Status == "online"
    }, 5*time.Second, 100*time.Millisecond, "device should
        be online within 5 seconds")
}
```

This document is a living artifact. It is updated with every significant change to the testing infrastructure, test coverage targets, or HelixConstitution amendments. All updates require review by both the Engineering Lead and the QA Lead.